C Pointers and Memory

Based on ECM2433 pointer, arrays/strings, functions-with-pointers, and dynamic-memory material.

Description: What You Need To Know

1. Pointers store addresses

The address-of operator produces an address. A pointer stores an address. Dereferencing a pointer accesses the object at that address.

```
int x = 4;
int *p = &x;
*p = 9;
printf("%d\n", x);  /* 9 */
```

2. Arrays, strings, and bounds

Arrays are contiguous memory. C does not reliably stop out-of-bounds access. A C string is a char array ending with a null terminator.

```
char s[] = "Hello";  /* H e l l o \0 */
printf("%zu\n", sizeof s);  /* 6 for this array */
printf("%zu\n", strlen(s)); /* 5 visible characters */
```

3. Passing pointers to functions

C passes arguments by value. To update the caller's object, pass its address and write through the pointer.

```
void update_integer(int *p) {
    *p = 42;
}

int value = 5;
update_integer(&value);
```

4. Dynamic memory

Use malloc for heap allocation, check for NULL, stay within bounds, and free each successful allocation exactly once.

```
int *a = malloc(n * sizeof *a);
if (a == NULL) {
    return 1;
}

for (int i = 0; i < n; i++) {
    a[i] = i + 1;
}

free(a);
a = NULL;
```

5. Common memory bugs

- Wild pointer: using a pointer that was never initialised.
- Dangling pointer: using a pointer after the object it pointed at no longer exists.
- Double-free: freeing the same allocation twice.
- Memory leak: losing the last pointer to allocated memory before freeing it.
- Out-of-bounds access: reading or writing outside the valid range.

Practice Questions

1. In the code `int x = 4; int *p = &x;`, identify the value, the address of x, the pointer value, and the dereferenced value.
2. Predict the output after `*p = *p + 3` when p points at x and x starts at 4.
3. Write a function that sets a caller-owned integer to 42 through a pointer parameter.
4. Write a pointer-based integer swap function.
5. For `char s[20] = "Hello";`, compare `sizeof s` and `strlen(s)`.
6. Explain why writing `array[15]` is unsafe when an array has 10 elements.
7. Write code that allocates n integers, checks allocation, fills them, and frees them.

8 Explain the bug in `char *p; strcpy(p, "Hello");` and give a safe fix.

9 Outline the allocation and cleanup pattern for a dynamic 2D array using `int **X`.

10 Classify these bugs: use after free, double free, forgotten free, and dereferencing an uninitialised pointer.

Mark Scheme

Use this after attempting the questions. Correct reasoning matters as much as final code.

1. `x` is the integer object; the address of `x` is produced with the address-of operator; `p` stores that address; `*p` is the integer stored there.
2. The output is `7`. The pointer refers to `x`, so assigning through it changes `x`.
3. Expected pattern: `void update_integer(int *p) { *p = 42; }`, called by passing the address of the caller's variable.
4. Expected body: `int tmp = *p; *p = *q; *q = tmp;`.
5. `sizeof s` is 20 bytes for the whole array. `strlen(s)` is 5 because it counts visible characters before the null terminator.
6. It writes outside the valid range. The behaviour is undefined: it may corrupt memory, crash, or appear to work.
7. Use `malloc(n * sizeof *a)`, check for `NULL`, loop over valid indexes, then `free(a)`.
8. The pointer is uninitialised and does not point at writable storage. Safe fixes include a large enough char array or allocated storage with a NULL check.
9. Allocate the row-pointer array, allocate each row, clean up earlier rows if a later row fails, then free rows and finally the row-pointer array.
10. Use-after-free/dangling pointer; double-free; memory leak; wild pointer or undefined behaviour.

Useful 2D Cleanup Pattern

```
int **X = malloc(R * sizeof *X);
if (X == NULL) {
    return 1;
}

for (int r = 0; r < R; r++) {
    X[r] = malloc(C * sizeof *X[r]);
    if (X[r] == NULL) {
        for (int i = 0; i < r; i++) {
            free(X[i]);
        }
        free(X);
        return 1;
    }
}

for (int r = 0; r < R; r++) {
    free(X[r]);
}
free(X);
```